

Integration Manual

for S32K1_S32M24X CAN_43_FLEXCAN Driver

Document Number: IM2CAN_43_FLEXCANASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Building the driver	6
3.1 Build Options	6
3.1.1 GCC Compiler/Assembler/Linker Options	6
3.1.1.1 GCC Compiler Options	6
3.1.1.2 GCC Assembler Options	8
3.1.1.3 GCC Linker Options	9
3.1.2 IAR Compiler/Assembler/Linker Options	9
3.1.2.1 IAR Compiler Options	9
3.1.2.2 IAR Assembler Options	10
3.1.2.3 IAR Linker Options	11
3.1.3 GHS Compiler/Assembler/Linker Options	11
3.1.3.1 GHS Compiler Options	11
3.1.3.2 GHS Assembler Options	13
3.1.3.3 GHS Linker Options	13
3.2 Files required for compilation	14
3.2.1 Files required for CAN Driver	14
3.2.1.1 Driver files	14
3.2.1.2 Driver Generated Files	14
3.2.2 Files required for FlexCAN Driver	15
3.2.2.1 Driver files	15
3.2.2.2 Generated Files	15
3.2.2.3 Preprocessor symbols	15
3.3 Setting up the plugins	15
3.3.1 Location of various files inside the CAN module folder	15
4 Function calls to module	16
4.1 Function Calls during Start-up	16
4.2 Function Calls during Shutdown	16
4.3 Function Calls during Wake-up	16
5 Module requirements	17
5.1 Exclusive areas to be defined in BSW scheduler	17
5.2 Exclusive areas not available on this platform	25
5.3 Peripheral Hardware Requirements	26
5.4 ISR to configure within AutosarOS - dependencies	26
5.5 ISR Macro	28
5.5.1 Without an Operating System	28
5.5.1.1 Using Software Vector Mode	28
5.5.1.2 Using Hardware Vector Mode	28
5.5.2 With an Operating System	28
5.6 Other AUTOSAR modules - dependencies	29
5.7 Data Cache Restrictions	29
5.8 User Mode support	29
5.8.1 User Mode configuration in the module	29
5.8.2 User Mode configuration in AutosarOS	32
5.9 MultiPartition support	32

6 Main API Requirements	34
6.1 Main function calls within BSW scheduler	34
6.2 API Requirements	34
6.3 Calls to Notification Functions, Callbacks, Callouts	35
7 Memory allocation	36
7.1 Sections to be defined in Can_43_FLEXCAN_MemMap.h	36
7.2 Linker command file	37
8 Integration Steps	38
9 External assumptions for driver	39



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This Integration Manual describes the integration requirements for NXP Semiconductor AUTOSAR CAN for S32K1_S32M24X microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64
- s32k146_lqfp64

- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Building the driver

- [Build Options](#)
- [Files required for compilation](#)
- [Setting up the plugins](#)

This section describes the source files and various compilers, linker options used for building the driver. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- [GCC Compiler/Assembler/Linker Options](#)
- [IAR Compiler/Assembler/Linker Options](#)
- [GHS Compiler/Assembler/Linker Options](#)

The RTD driver files are compiled using:

- NXP GCC 10.2.0 20200723 (Build 1728 Revision g5963bc8)
- IAR ANSI C/C++ Compiler V8.40.3.228/W32 for ARM Functional Safety
- Green Hills Multi 7.1.6d / Compiler 2020.1.4

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D2M30I0R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D2 identifies S32K1 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)

Compiler Option	Description
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x or S32M24x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x or S32M24x devices)
-mfpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K11x devices)
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioned -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned

Compiler Option	Description
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↔ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↔ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32↔M24x devices)
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↔ as a macro, with definition 1. Allows drivers to be configured in user mode.
-sysroot=	Specifies the path to the sysroot, for Cortex-M7 it is /arm-none-eabi/newlib
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-xassembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mfpu=fpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x devices)
-mfpu=auto	Specifies the floating-point hardware available on the target (for S32K11x devices)

Assembler Option	Description
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x or S32M24x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x or S32M24x devices)
-mfpu=auto	Specifies the floating-point hardware available on the target (for S32K11x devices)
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library
-n	Turn off page alignment of sections, and disable linking against shared libraries
-sysroot=	Specifies the path to the sysroot, for Cortex-M7 it is /arm-none-eabi/newlib
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.2 IAR Compiler/Assembler/Linker Options

3.1.2.1 IAR Compiler Options

Compiler Option	Description
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)

Compiler Option	Description
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-cpu_mode=thumb	Generates code that executes in Thumb state
-endian=little	Generate code for a processor running in little-endian mode
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x or S32M24x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-e	Enables all IAR C language extensions
-Ohz	Optimize for size. The compiler will emit AEABI attributes indicating the requested optimization goal. This information can be used by the linker to select smaller or faster variants of DLIB library functions
-debug	Makes the compiler include debugging information in the object modules. Including debug information will make the object files larger
-no_clustering	Disables static clustering optimizations. Static and global variables defined within the same module will not be arranged so that variables that are accessed in the same function are close to each other
-no_mem_idioms	Makes the compiler not optimize certain memory access patterns
-no_explicit_zero_opt	Do not treat explicit initializations to zero of static variables as zero initializations
-require_prototypes	Force the compiler to verify that all functions have proper prototypes. Generates an error otherwise
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages
-diag_suppress=Pa050	Suppresses diagnostic message Pa050
-DIAR	Predefine IAR as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode.
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.2.2 IAR Assembler Options

Assembler Option	Description
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-cpu_mode thumb	Selects the thumb mode for the assembler directive CODE
-g	Disables the automatic search for system include files
-r	Generates debug information

3.1.2.3 IAR Linker Options

Linker Option	Description
-map filename	Produces a map file
-config linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x or S32M24x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-entry __start	Treats __start as a root symbol and start label
-enable_stack_usage	Enables stack usage analysis. If a linker map file is produced, a stack usage chapter is included in the map file
-skip_dynamic_initialization	Dynamic initialization (typically initialization of C++ objects with static storage duration) will not be performed automatically during application startup
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages

3.1.3 GHS Compiler/Assembler/Linker Options

3.1.3.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm4	Selects target processor: Arm Cortex M4 (for S32K14x or S32M24x devices)
-cpu=cortexm0plus	Selects target processor: Arm Cortex M0+ (for S32K11x devices)
-thumb	Selects generating code that executes in Thumb state

Compiler Option	Description
-fpu=vfpv4_d16	Specifies hardware floating-point using the v4 version of the VFP instruction set, with 16 double-precision floating-point registers (for S32K14x or S32M24x devices)
-fsingle	Use hardware single-precision, software double-precision FP instructions (for S32K14x or S32M24x devices)
-fsoft	Specifies software floating-point (SFP) mode. This setting causes your target to use integer registers to hold floating-point data and use library subroutine calls to emulate floating-point operations (for S32K11x devices)
-C99	Use (strict ISO) C99 standard (without extensions)
-ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
-gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
-unsigned_chars	Let the type char be unsigned, like unsigned char
-unsigned_fields	Bitfields declared with an integer type are unsigned
-no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
-no_exceptions	Disables C++ support for exception handling
-no_slash_comment	C++ style // comments are not accepted and generate errors
-prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
-incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)

Compiler Option	Description
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.3.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm4	Selects target processor: Arm Cortex M4 (for S32K14x or S32M24x devices)
-cpu=cortexm0plus	Selects target processor: Arm Cortex M0+ (for S32K11x devices)
-fpu=vfpv4_d16	Specifies hardware floating-point using the v4 version of the VFP instruction set, with 16 double-precision floating-point registers (for S32K14x devices)
-fsingle	Use hardware single-precision, software double-precision FP instructions (for S32K14x devices)
-fsoft	Specifies software floating-point (SFP) mode. This setting causes your target to use integer registers to hold floating-point data and use library subroutine calls to emulate floating-point operations (for S32K11x devices)
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.3.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for the modules in this library is provided in the src/libstartup directory

Linker Option	Description
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library (for S32K14x or S32M24x devices)
-lind_sf	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library (for S32K11x devices)
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-keep=C40_Ip_AccessCode	Avoid linker remove function C40_Ip_AccessCode from Fls module because it is not referenced explicitly
-nostartfiles	Controls the start files to be linked into the executable

3.2 Files required for compilation

This section describes the include files required to compile, assemble (if assembler code) and link the CAN driver. To avoid integration of incompatible files, all the include files from other modules shall have the same `AR_MAJOR_VERSION` and `AR_MINOR_VERSION`, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

3.2.1 Files required for CAN Driver

3.2.1.1 Driver files

```

${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\Can_43_FLEXCAN.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\Can_43_FLEXCAN_Ipw.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\Can_43_FLEXCAN_Ipw_Irq.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\FlexCAN_Ip.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\FlexCAN_Ip_Irq.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\FlexCAN_Ip_Hw_Access.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\Can_43_FLEXCAN.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\Can_43_FLEXCAN_Flexcan_Types.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\Can_43_FLEXCAN_Ipw.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\Can_43_FLEXCAN_Ipw_Irq.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\Can_43_FLEXCAN_Ipw_Types.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\Can_43_FLEXCAN_Irq.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_DeviceReg.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Hw_Access.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Irq.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Types.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Wrapper.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_TrustedFunctions.h

```

3.2.1.2 Driver Generated Files

The following files must be generated by the user using a configuration tool:

- Can_43_FLEXCAN_Cfg.h
- Can_43_FLEXCAN_Ipw_Cfg.h
- Can_43_FLEXCAN_<VariantName>_PBcfg.h
- Can_43_FLEXCAN_Ipw_<VariantName>_PBcfg.h
- Can_43_FLEXCAN_<VariantName>_PBcfg.c
- Can_43_FLEXCAN_Ipw_<VariantName>_PBcfg.c
- FlexCAN_Ip_Cfg.h
- FlexCAN_Ip_CfgDefines.h
- FlexCAN_Ip_<VariantName>_PBcfg.h
- FlexCAN_Ip_<VariantName>_PBcfg.c

3.2.2 Files required for FlexCAN Driver

3.2.2.1 Driver files

The following files need to be compiled in the project:

```

${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\FlexCAN_Ip.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\FlexCAN_Ip_HwAccess.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\src\FlexCAN_Ip_Irq.c
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_DeviceReg.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Hw_Access.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Irq.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Types.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_Wrapper.h
${S32RTD_PATH}\Can_43_FLEXCAN_TS_T40D2M30I0R0\include\FlexCAN_Ip_TrustedFunctions.h

```

3.2.2.2 Generated Files

The following files must be generated by the user using a configuration tool:

```

- FlexCAN_Ip_Cfg.h
- FlexCAN_Ip_CfgDefines.h
- FlexCAN_Ip_<FunctionalGroup>_PBcfg.h
- FlexCAN_Ip_<FunctionalGroup>_PBcfg.c

```

3.2.2.3 Preprocessor symbols

No special symbols are required for this component.

3.3 Setting up the plugins

The CAN driver was designed to be configured by using the EB Tresos Studio (version EB tresos Studio 29.0.0 or later.)

3.3.1 Location of various files inside the CAN module folder

VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format:

```
- Can_43_FLEXCAN_<asr_package_name>\config\Can_43_FLEXCAN.xdm
```

VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:

```
- Can_43_FLEXCAN_<asr_package_name>\autosar\Can_43_FLEXCAN_<subderivative_name>.epd
```

Chapter 4

Function calls to module

- [Function Calls during Start-up](#)
- [Function Calls during Shutdown](#)
- [Function Calls during Wake-up](#)

4.1 Function Calls during Start-up

The CAN module shall be initialized by `Can_43_FLEXCAN_Init()` service call during the start-up. API service `Can_43_FLEXCAN_SetControllerMode(Can_Controller, CAN_CS_STARTED)` shall be used for setting the CAN controller to running mode.

/note Can driver don't enable FlexCAN instances clocks, in order the user must enable the clocks from Clock Module Configuration for the instances used. Pin settings are not related to Can driver or plugin configuration. GPIO pins used for connection of CAN physical layer have to be properly assigned to the IPV_FlexCAN module prior the CAN initialization.

4.2 Function Calls during Shutdown

The IPV_FlexCAN IP has many Low Power Modes:

- **Freeze Mode** This low power mode is entered when the HALT and FRZ bits in the MCR Register are asserted. Module ignores the Rx input pin and drives the Tx pin as recessive, stops the prescaler, thus halting all CAN protocol activities and grants write access to the Error Counters Register (ECR), which is read-only in other modes. Exit from this mode is done by negating the FRZ and HALT bits in the MCR Register or when the MCU is removed from Debug Mode /note It is not possible to exit from this mode by receiving a message on the Can bus.
- **Module Disable Mode** This low power mode is entered when the MDIS bit in the MCR Register is asserted. Module shuts down the clocks to the CAN Protocol Interface and Message Buffer Management sub-modules. Exit from this mode is done by negating the MDIS bit in the MCR Register.

/note It is not possible to exit from this mode by receiving a message on the Can bus.

4.3 Function Calls during Wake-up

Hardware does not support wake-up.

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- Exclusive areas not available on this platform
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- MultiPartition support

5.1 Exclusive areas to be defined in BSW scheduler

CAN_EXCLUSIVE_AREA_00 is used in function Can_43_FLEXCAN_DisableControllerInterrupts to protect the variable Can_u8DisableInterruptLevel.

CAN_EXCLUSIVE_AREA_01 is used in function Can_43_FLEXCAN_EnableControllerInterrupts to protect the variable Can_u8DisableInterruptLevel.

CAN_EXCLUSIVE_AREA_02 is used in function Can_43_FLEXCAN_SetBaudrate to protect the register CAN_MCR in FlexCAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_02 is used in function Can_43_FLEXCAN_SetControllerMode to protect the register CAN_MCR in FlexCAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_02 is used in function Can_43_FLEXCAN_Init to protect the register CAN_MCR in FlexCAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_02 is used in function Can_43_FLEXCAN_DeInit to protect the register CAN_MCR in FlexCAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_03 is used in function Can_43_FLEXCAN_ListenOnlyMode to protect the register CAN_MCR in FlexCAN_Enable.

CAN_EXCLUSIVE_AREA_03 is used in function Can_43_FLEXCAN_EnableControllerInterrupts to protect the register CAN_MCR in FlexCAN_Enable.

CAN_EXCLUSIVE_AREA_03 is used in function Can_43_FLEXCAN_DisableControllerInterrupts to protect the register CAN_MCR in FlexCAN_Enable.

Module requirements

CAN_EXCLUSIVE_AREA_03 is used in function `Can_43_FLEXCAN_SetClockMode` to protect the register `CAN_MCR` in `FlexCAN_Enable`.

CAN_EXCLUSIVE_AREA_04 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_MCR` in `FlexCAN_ExitFreezeMode`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_ListenOnlyMode` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_EnableControllerInterrupts` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_DisableControllerInterrupts` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_SetBaudrate` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_Init` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_SetClockMode` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_05 is used in function `Can_43_FLEXCAN_DeInit` to protect the register `CAN_MCR` in `FlexCAN_Disable`.

CAN_EXCLUSIVE_AREA_06 is used in function `Can_43_FLEXCAN_EnableControllerInterrupts` to protect the register `CAN_MCR`, `CAN_CTRL1`, `CAN_CTRL2` in `FlexCAN_SetErrIntCmd`.

CAN_EXCLUSIVE_AREA_06 is used in function `Can_43_FLEXCAN_DisableControllerInterrupts` to protect the register `CAN_MCR`, `CAN_CTRL1`, `CAN_CTRL2` in `FlexCAN_SetErrIntCmd`.

CAN_EXCLUSIVE_AREA_06 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_MCR`, `CAN_CTRL1`, `CAN_CTRL2` in `FlexCAN_SetErrIntCmd`.

CAN_EXCLUSIVE_AREA_07 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_MCR` in `FlexCAN_Ip_SetStartMode`.

CAN_EXCLUSIVE_AREA_08 is used in function `Can_43_FLEXCAN_Init` to protect the register `CAN_MCR` in `FlexCAN_Ip_SetRxMaskType`.

CAN_EXCLUSIVE_AREA_08 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_MCR` in `FlexCAN_Ip_SetRxMaskType`.

CAN_EXCLUSIVE_AREA_10 is used in function `Can_43_FLEXCAN_ListenOnlyMode` to protect the register `CAN_CTRL1` in `FlexCAN_Ip_SetListenOnlyMode`.

CAN_EXCLUSIVE_AREA_10 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_CTRL1` in `FlexCAN_Ip_SetListenOnlyMode`.

CAN_EXCLUSIVE_AREA_11 is used in function `Can_43_FLEXCAN_AbortMb` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FLEXCAN_ClearMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_11 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FLEXCAN_ClearMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_11 is used in function `Can_43_FLEXCAN_ErrorIrqCallback` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FLEXCAN_ClearMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_13 is used in function `Can_43_FLEXCAN_Init` to protect the register `CAN_MCR` in `FlexCAN_SetRxFifoFilter`.

CAN_EXCLUSIVE_AREA_13 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_MCR` in `FlexCAN_SetRxFifoFilter`.

CAN_EXCLUSIVE_AREA_14 is used in function `Can_43_FLEXCAN_SetClockMode` to protect the register `CAN_CTRL1` in `FlexCAN_Ip_SetBaudrate`.

CAN_EXCLUSIVE_AREA_14 is used in function `Can_43_FLEXCAN_SetBaudrate` to protect the register `CAN_CTRL1` in `FlexCAN_Ip_SetBaudrate`.

CAN_EXCLUSIVE_AREA_14 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_CTRL1` in `FlexCAN_Ip_SetBaudrate`.

CAN_EXCLUSIVE_AREA_15 is used in function `Can_43_FLEXCAN_SetClockMode` to protect the register `CAN_MCR`, `CAN_FDCTRL` in `FlexCAN_Ip_SetBaudrateCbt`.

CAN_EXCLUSIVE_AREA_15 is used in function `Can_43_FLEXCAN_SetBaudrate` to protect the register `CAN_MCR`, `CAN_FDCTRL` in `FlexCAN_Ip_SetBaudrateCbt`.

CAN_EXCLUSIVE_AREA_15 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_MCR`, `CAN_FDCTRL` in `FlexCAN_Ip_SetBaudrateCbt`.

CAN_EXCLUSIVE_AREA_16 is used in function `Can_43_FLEXCAN_Init` to protect the register `CAN_FDCTRL` in `FlexCAN_Ip_SetTDCOffset`.

CAN_EXCLUSIVE_AREA_16 is used in function `Can_43_FLEXCAN_SetBaudrate` to protect the register `CAN_FDCTRL` in `FlexCAN_Ip_SetTDCOffset`.

CAN_EXCLUSIVE_AREA_16 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_FDCTRL` in `FlexCAN_Ip_SetTDCOffset`.

CAN_EXCLUSIVE_AREA_17 is used in function `Can_43_FLEXCAN_Init` to protect the register `CAN_CTRL2` in `FlexCAN_Ip_SetTxArbitrationStartDelay`.

CAN_EXCLUSIVE_AREA_17 is used in function `Can_43_FLEXCAN_SetBaudrate` to protect the register `CAN_CTRL2` in `FlexCAN_Ip_SetTxArbitrationStartDelay`.

CAN_EXCLUSIVE_AREA_17 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the register `CAN_CTRL2` in `FlexCAN_Ip_SetTxArbitrationStartDelay`.

CAN_EXCLUSIVE_AREA_18 is used in function `Can_43_FLEXCAN_Write` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN0_ORED_0_31_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN0_ORED_0_15_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN0_ORED_16_31_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN0_ORED_32_47_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN0_ORED_48_63_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN1_ORED_0_15_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

Module requirements

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN1_ORED_16_31_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN1_ORED_32_47_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN1_ORED_48_63_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN2_ORED_0_15_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `ISR(CAN2_ORED_16_31_MB_IRQHandler)` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

CAN_EXCLUSIVE_AREA_18 is used in function `Can_43_FLEXCAN_SetControllerMode` to protect the variable `g_FlexCAN_u32ImaskBuff` in `FlexCAN_SetMsgBuffIntCmd`.

Exclusive Areas implemented in Low level driver layer (IPL)

CAN_EXCLUSIVE_AREA_02 is used in function `FlexCAN_EnterFreezeMode` to protect the updates for:

- `CAN_MCR` register

CAN_EXCLUSIVE_AREA_03 is used in function `FlexCAN_Enable` to protect the updates for:

- `CAN_MCR` register

CAN_EXCLUSIVE_AREA_04 is used in function `FlexCAN_ExitFreezeMode` to protect the updates for:

- `CAN_MCR` register

CAN_EXCLUSIVE_AREA_05 is used in function `FlexCAN_Disable` to protect the updates for:

- `CAN_MCR` register

CAN_EXCLUSIVE_AREA_06 is used in function `FlexCAN_SetErrIntCmd` to protect the updates for:

- `CAN_MCR`, `CAN_CTRL1`, `CAN_CTRL2` register

CAN_EXCLUSIVE_AREA_07 is used in function `FlexCAN_Ip_SetStartMode` to protect the updates for:

- `CAN_MCR` register

CAN_EXCLUSIVE_AREA_08 is used in function `FlexCAN_Ip_SetRxMaskType` to protect the updates for:

- `CAN_MCR` register

CAN_EXCLUSIVE_AREA_09 is used in function `FlexCAN_Ip_ClearTDCFail` to protect the updates for:

- `CAN_FDCTRL` register

CAN_EXCLUSIVE_AREA_10 is used in function `FlexCAN_Ip_SetListenOnlyMode` to protect the updates for:

- `CAN_CTRL1` register

CAN_EXCLUSIVE_AREA_11 is used in function `FLEXCAN_ClearMsgBuffIntCmd` to protect the updates for:

- `g_FlexCAN_u32ImaskBuff` variable

CAN_EXCLUSIVE_AREA_12 is used in function `FlexCAN_Ip_ConfigPN_Privileged` to protect the updates for:

- CAN_MCR register
CAN_EXCLUSIVE_AREA_13 is used in function FlexCAN_SetRxFifoFilter to protect the updates for:
- CAN_MCR register
CAN_EXCLUSIVE_AREA_14 is used in function FlexCAN_Ip_SetBtrRate to protect the updates for:
- CAN_CTRL1 register
CAN_EXCLUSIVE_AREA_15 is used in function FlexCAN_Ip_SetBtrRateCbt to protect the updates for:
- CAN_MCR, CAN_FDCTRL register
CAN_EXCLUSIVE_AREA_16 is used in function FlexCAN_Ip_SetTDCOffset to protect the updates for:
- CAN_FDCTRL register
CAN_EXCLUSIVE_AREA_17 is used in function FlexCAN_Ip_SetTxArbitrationStartDelay to protect the updates for:
- CAN_CTRL2 register
CAN_EXCLUSIVE_AREA_18 is used in function FlexCAN_SetMsgBuffIntCmd to protect the updates for:
- g_FlexCAN_u32ImaskBuff variable
CAN_EXCLUSIVE_AREA_19 is used in function FlexCAN_Ip_ConfigTimeStamp_Privileged to protect the updates for:
- CAN_CTRL2 register
CAN_EXCLUSIVE_AREA_20 is used in function FlexCAN_Ip_ManualBusOffRecovery to protect the updates for:
- CAN_CTRL1 register

Critical Region Exclusive Matrix Below is the table depicting the exclusivity between different critical region IDs from the CAN driver. If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

The critical regions from Tx/Rx interrupts are grouped in “ISRs Critical Regions (composed diagram)”. If an exclusive area is “exclusive” with the composed “ISRs Critical Regions (composed diagram)” group, it means that it is exclusive with each one of the ISR critical regions.

Table 5.1 Exclusive Areas

#	A← R← E← A← ← 00	A← R← E← A← ← 01	A← R← E← A← ← 02	A← R← E← A← ← 03	A← R← E← A← ← 04	A← R← E← A← ← 05	A← R← E← A← ← 06	A← R← E← A← ← 07	A← R← E← A← ← 08	A← R← E← A← ← 09	A← R← E← A← ← 10	A← R← E← A← ← 11	A← R← E← A← ← 12	A← R← E← A← ← 13	A← R← E← A← ← 14	A← R← E← A← ← 15	A← R← E← A← ← 16	A← R← E← A← ← 17	A← R← E← A← ← 18	A← R← E← A← ← 19	A← R← E← A← ← 20	I← S← Rs Crit- i- cal Re- gions (com- posed di- a- gram)
A← R← E← A← ← 00		x																				
A← R← E← A← ← 01	x																					
A← R← E← A← ← 02				x	x	x	x	x	x				x	x		x						
A← R← E← A← ← 03			x		x	x	x	x	x				x	x		x						
A← R← E← A← ← 04			x	x		x	x	x	x				x	x		x						
A← R← E← A← ← 05			x	x	x		x	x	x				x	x		x						

#	A← R← E← A← _← 00	A← R← E← A← _← 01	A← R← E← A← _← 02	A← R← E← A← _← 03	A← R← E← A← _← 04	A← R← E← A← _← 05	A← R← E← A← _← 06	A← R← E← A← _← 07	A← R← E← A← _← 08	A← R← E← A← _← 09	A← R← E← A← _← 10	A← R← E← A← _← 11	A← R← E← A← _← 12	A← R← E← A← _← 13	A← R← E← A← _← 14	A← R← E← A← _← 15	A← R← E← A← _← 16	A← R← E← A← _← 17	A← R← E← A← _← 18	A← R← E← A← _← 19	A← R← E← A← _← 20	I← S← Rs Crit- i- cal Re- gions (com- posed di- a- gram)
A← R← E← A← _← 06			x	x	x	x		x	x		x		x	x	x	x		x		x	x	
A← R← E← A← _← 07			x	x	x	x	x		x				x	x		x						
A← R← E← A← _← 08			x	x	x	x	x	x					x	x		x						
A← R← E← A← _← 09																x	x					
A← R← E← A← _← 10							x								x						x	
A← R← E← A← _← 11																		x				

Module requirements

#	A← R← E← A← _← 00	A← R← E← A← _← 01	A← R← E← A← _← 02	A← R← E← A← _← 03	A← R← E← A← _← 04	A← R← E← A← _← 05	A← R← E← A← _← 06	A← R← E← A← _← 07	A← R← E← A← _← 08	A← R← E← A← _← 09	A← R← E← A← _← 10	A← R← E← A← _← 11	A← R← E← A← _← 12	A← R← E← A← _← 13	A← R← E← A← _← 14	A← R← E← A← _← 15	A← R← E← A← _← 16	A← R← E← A← _← 17	A← R← E← A← _← 18	A← R← E← A← _← 19	A← R← E← A← _← 20	I← S← Rs Crit- i- cal Re- gions (com- posed di- a- gram)	
A← R← E← A← _← 12			x	x	x	x	x	x	x					x		x							
A← R← E← A← _← 13			x	x	x	x	x	x	x				x			x							
A← R← E← A← _← 14							x				x					x						x	
A← R← E← A← _← 15			x	x	x	x	x	x	x	x			x	x	x		x						
A← R← E← A← _← 16										x						x							
A← R← E← A← _← 17							x													x			

#	A← R← E← A← _← 00	A← R← E← A← _← 01	A← R← E← A← _← 02	A← R← E← A← _← 03	A← R← E← A← _← 04	A← R← E← A← _← 05	A← R← E← A← _← 06	A← R← E← A← _← 07	A← R← E← A← _← 08	A← R← E← A← _← 09	A← R← E← A← _← 10	A← R← E← A← _← 11	A← R← E← A← _← 12	A← R← E← A← _← 13	A← R← E← A← _← 14	A← R← E← A← _← 15	A← R← E← A← _← 16	A← R← E← A← _← 17	A← R← E← A← _← 18	A← R← E← A← _← 19	A← R← E← A← _← 20	I← S← Rs Crit- i- cal Re- gions (com- posed di- a- gram)
A← R← E← A← _← 18											x											x
A← R← E← A← _← 19						x												x				
A← R← E← A← _← 20						x				x					x							
I← S← Rs Crit- i- cal Re- gions (com- posed di- a- gram)																		x				

5.2 Exclusive areas not available on this platform

CAN_EXCLUSIVE_AREA_21 is not available. CAN_EXCLUSIVE_AREA_22 is not available.

5.3 Peripheral Hardware Requirements

peripheral hardware requirements template.

5.4 ISR to configure within AutosarOS - dependencies

The following ISR's are used by the CAN driver:

- Table with interrupts for S32K11X

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	26
CAN0_ORED_0_31_MB_IRQHandler	27

- Table with interrupts for S32K142

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	94
CAN0_Error_IRQHandler	95
CAN0_Wake_Up_IRQHandler	96
CAN0_ORED_0_15_MB_IRQHandler	97
CAN0_ORED_16_31_MB_IRQHandler	98
CAN1_ORED_IRQHandler	101
CAN1_Error_IRQHandler	102
CAN1_ORED_0_15_MB_IRQHandler	104

- Table with interrupts for S32K144

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	94
CAN0_Error_IRQHandler	95
CAN0_Wake_Up_IRQHandler	96
CAN0_ORED_0_15_MB_IRQHandler	97
CAN0_ORED_16_31_MB_IRQHandler	98
CAN1_ORED_IRQHandler	101
CAN1_Error_IRQHandler	102
CAN1_ORED_0_15_MB_IRQHandler	104
CAN2_ORED_IRQHandler	108
CAN2_Error_IRQHandler	109
CAN2_ORED_0_15_MB_IRQHandler	111

- Table with interrupts for S32K144W

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	94
CAN0_Error_IRQHandler	95

ISR Name	Hardware Interrupt Vector
CAN0_Wake_Up_IRQHandler	96
CAN0_ORED_0_15_MB_IRQHandler	97
CAN0_ORED_16_31_MB_IRQHandler	98
CAN0_ORED_32_47_MB_IRQHandler	99
CAN0_ORED_48_63_MB_IRQHandler	100
CAN1_ORED_IRQHandler	101
CAN1_Error_IRQHandler	102
CAN1_ORED_0_15_MB_IRQHandler	104
CAN1_ORED_16_31_MB_IRQHandler	105
CAN1_ORED_32_47_MB_IRQHandler	106
CAN1_ORED_48_63_MB_IRQHandler	107

- Table with interrupts for S32K146

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	94
CAN0_Error_IRQHandler	95
CAN0_Wake_Up_IRQHandler	96
CAN0_ORED_0_15_MB_IRQHandler	97
CAN0_ORED_16_31_MB_IRQHandler	98
CAN1_ORED_IRQHandler	101
CAN1_Error_IRQHandler	102
CAN1_ORED_0_15_MB_IRQHandler	104
CAN1_ORED_16_31_MB_IRQHandler	105
CAN2_ORED_IRQHandler	108
CAN2_Error_IRQHandler	109
CAN2_ORED_0_15_MB_IRQHandler	111

- Table with interrupts for S32K148

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	94
CAN0_Error_IRQHandler	95
CAN0_Wake_Up_IRQHandler	96
CAN0_ORED_0_15_MB_IRQHandler	97
CAN0_ORED_16_31_MB_IRQHandler	98
CAN1_ORED_IRQHandler	101
CAN1_Error_IRQHandler	102
CAN1_ORED_0_15_MB_IRQHandler	104
CAN1_ORED_16_31_MB_IRQHandler	105
CAN2_ORED_IRQHandler	108
CAN2_Error_IRQHandler	109
CAN2_ORED_0_15_MB_IRQHandler	111

ISR Name	Hardware Interrupt Vector
CAN2_ORED_16_31_MB_IRQHandler	112

- Table with interrupts for S32M241 and S32M242

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	78
CAN0_Error_IRQHandler	79
CAN0_Wake_Up_IRQHandler	80
CAN0_ORED_0_15_MB_IRQHandler	81
CAN0_ORED_16_31_MB_IRQHandler	82

- Table with interrupts for S32M243 and S32M244

ISR Name	Hardware Interrupt Vector
CAN0_ORED_IRQHandler	78
CAN0_Error_IRQHandler	79
CAN0_Wake_Up_IRQHandler	80
CAN0_ORED_0_15_MB_IRQHandler	81
CAN0_ORED_16_31_MB_IRQHandler	82
CAN0_ORED_32_47_MB_IRQHandler	83
CAN0_ORED_48_63_MB_IRQHandler	84

5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.5.1 Without an Operating System

The macro `USING_OS_AUTOSAROS` must not be defined.

5.5.1.1 Using Software Vector Mode

The macro `USE_SW_VECTOR_MODE` must be defined and the ISR macro is defined as: `#define ISR(IsrName) void IsrName(void)`

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.5.1.2 Using Hardware Vector Mode

The macro `USE_SW_VECTOR_MODE` must not be defined and the ISR macro is defined as: `#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)`

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.5.2 With an Operating System

Please refer to your OS documentation for description of the ISR macro.

5.6 Other AUTOSAR modules - dependencies

- **BaseNXP**: The Base module contains the common files/definitions needed by all MCAL modules.
- **Mcu**: The Mcu driver provides services for basic microcontroller initialization, power down functionality, reset and microcontroller specific functions required by other MCAL software modules. The clocks need to be initialized prior to using the Can driver. The clock frequency may affect the Can bit rate, the transmitting or receiving a Can frame process.
- **Port**: The Port module is used to configure the port pins with the needed modes, before they are used by the Can module.
- **EcuC** : The ECUC module is used for ECU configuration. Can modules need ECUC to retrieve the variant information.
- **Det** The Det module is used for enabling Default error detection. The API function used is Det_ReportError(). The activation / deactivation of Default error detection is configurable using the 'CanDevErrorDetect' configuration parameter.
- **Resource**: Sub-Derivative model is selected from Resource configuration.
- **Rte**: The Rte module is needed for implementing data consistency of exclusive areas that are used by Can module.
- **EcuM**: This module is used for processing the Wakeup notifications of CAN. Whenever the module is in 'Sleep' mode and a wakeup event occurs, it is reported to EcuM through the EcuM_CheckWakeupEvent() API.
- **Mcl**: This module is used for enabling DMA channels for Can controllers.
- **Platform**: This module is used to configure the interrupts for Can module when using interrupt method.
- **Rm**: This module is used to allocate and manage the hardware resource. Configuring DMA MUX for DMA channels and FlexCAN Peripheral.

5.7 Data Cache Restrictions

In the DMA transfer mode, DMA transfers may issue cache coherency problems. To avoid possible coherency issues when D-CACHE is enabled, the user shall ensure that the buffers used as TCD source and destination are allocated in the NON-CACHEABLE area (by means of Can_43_FLEXCAN_Memmap).

5.8 User Mode support

- [User Mode configuration in the module](#)
- [User Mode configuration in AutosarOS](#)

5.8.1 User Mode configuration in the module

The Can Driver can be run in user mode as long as the configuration parameter CanEnableUserModeSupport is enabled and MCAL_ENABLE_USER_MODE_SUPPORT is defined and call the following functions as trusted functions:

Function syntax	Description	Available via
void FlexCAN_ClearOutputLegacyFIFO(FLEXCAN_Type * base)	Clears Legacy RxFifo message buffers	FlexCAN_Ip_TrustedFunctions.h

Module requirements

Function syntax	Description	Available via
Flexcan_Ip_StatusType FlexCAN_Ip_Init_Privileged(uint8 Flexcan_Ip_u8Instance, Flexcan_Ip_StateType * Flexcan_Ip_pState, const Flexcan_Ip_ConfigType * Flexcan_Ip_pData)	Initializes the FlexCAN peripheral	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_ConfigRxFifo_Privileged(uint8 instance, Flexcan_Ip_RxFifoIdElementFormatType id_format, const Flexcan_Ip_IdTableType * id_filter_table)	FlexCAN Rx Legacy FIFO filter configuration	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_MainFunctionBusOff_Privileged(uint8 instance)	Check a bus-off event	FlexCAN_Ip_TrustedFunctions.h
boolean FlexCAN_Ip_GetStopMode_Privileged(uint8 instance)	Check if the FlexCAN instance is STOPPED	FlexCAN_Ip_TrustedFunctions.h
boolean FlexCAN_Ip_GetStartMode_Privileged(uint8 instance)	Check if the FlexCAN instance is STARTED	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_EnterFreezeMode_Privileged(uint8 instance)	Enter FlexCAN Module in Freeze Mode	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_ExitFreezeMode_Privileged(uint8 instance)	Exit FlexCAN Module from Freeze Mode	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetRxFifoGlobalMask_Privileged(uint8 instance, uint32 mask)	Set RxFifo Global Mask	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_Deinit_Privileged(uint8 instance)	DeInitialize the FlexCAN instance driver	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetStartMode_Privileged(uint8 instance)	Set the FlexCAN instance in START mode	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetStopMode_Privileged(uint8 instance)	Set the FlexCAN instance in STOP mode	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetRxMaskType_Privileged(uint8 instance, Flexcan_Ip_RxMaskType type)	Set RX masking type	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetRxMb14Mask_Privileged(uint8 instance, uint32 mask)	Set Rx14Mask filter for message buffer 14	FlexCAN_Ip_TrustedFunctions.h

Function syntax	Description	Available via
Flexcan_Ip_StatusType FlexCAN_Ip_SetRxMb15Mask_Privileged(uint8 instance, uint32 mask)	Set Rx14Mask filter for message buffer 15	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetRxIndividualMask_Privileged(uint8 instance, uint8 mb_idx, uint32 mask)	Sets the FlexCAN Rx individual mask	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetRxMbGlobalMask_Privileged(uint8 instance, uint32 mask)	Sets the FlexCAN Rx Message Buffer Global mask	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetBtrRate_Privileged(uint8 instance, const Flexcan_Ip_TimeSegmentType * bitrate, boolean enhExt)	Sets the FlexCAN bit rate for standard frames or the arbitration phase of FD frames	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetBtrRateCbt_Privileged(uint8 instance, const Flexcan_Ip_TimeSegmentType * bitrate, boolean bitRateSwitch)	Sets the FlexCAN bit rate for the data phase of FD frames	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetTxArbitrationStartDelay_Privileged(uint8 instance, uint8 value)	This function will set how many CAN bits the Tx arbitration process start point can	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetTDCOffset_Privileged(uint8 instance, boolean enable, uint8 offset)	Enables/Disables the Transceiver Delay Compensation feature and sets the Transceiver Delay Compensation Offset	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_EnableInterrupts_Privileged(uint8 u8Instance)	Enable all mb interrupts configured	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_DisableInterrupts_Privileged(uint8 u8Instance)	Disable all mb interrupts configured	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetErrorInt_Privileged(uint8 u8Instance, Flexcan_Ip_ErrorIntType type, boolean enable)	Enable\Disable Error or BusOff Interrupt	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetListenOnlyMode_Privileged(uint8 instance, const boolean enable)	Set FlexCAN Listen Only	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_ConfigTimeStamp_Privileged(uint8 instance, const Flexcan_Ip_TimeStampConfigType * time_stamp)	Set FlexCAN Config Timestamp	FlexCAN_Ip_TrustedFunctions.h

Module requirements

Function syntax	Description	Available via
Flexcan_Ip_StatusType FlexCAN_Ip_N_Ip_ConfigPN_Privileged(uint8 u8Instance, boolean bEnable, const Flexcan_Ip_PnConfigType * pPnConfig)	Configures Pretended Networking settings	FlexCAN_Ip_TrustedFunctions.h

5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may have the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_Ip_TrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter [User Mode configuration in the module](#) for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

`Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)`

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

```
#define OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)
```

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.
- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will un-marshalling of the parameters to call `<Function_Name>()` of driver. The `<Function_Name>()` functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

5.9 MultiPartition support

The Can implements the "MCAL Distribution" according to type II, in which the mappable element is set to HW Unit. For additional details, please refer to `AUTOSAR_EXP_BSWDistributionGuide`.

The Can and the mappable elements can be allocated to zero, one or several ECUC partitions, by means of "CanEcucPartionRef". If the Can is mapped to zero ECUC partitions, the Can behavior reverts to single-core implementation, similar to previous Autosar versions. If the Can is mapped to one or more ECUC partitions, the Can enforces the following multi-partition assumptions:

- The Can assumes there is a single EcucPartition allocated per core. Internally, the module will use the PartitionID returned by `GetUserId` API to reference the appropriate global data and configuration elements.
- The Can assumes the EcucCoreIDs are defined in a compact/consecutive order, starting from zero. The rationale is that the number of EcucPartitions is used for dimensioning the Can internal variables and the EcucCoreIDs are used for indexing those variables. (AR-86601 Zero based and dense IDs for OS-Cores and OSApplications)

- The Can assumes that initialization is performed on each core, Can_43_FLEXCAN_Init() is called separately for each core, using a different configuration structure. (Type II)
- The Can initialization expects the upper layer will pass the correct initialization pointer, specific to the partition in which the driver is to be used. For example: EcucPartition_1 is assigned to CoreID 1; Can_43_FLEXCAN_Init() function will be called with Can_43_FLEXCAN_Config_EcucPartition_1 configuration structure, on Core 1.
- The Can will check upon each API call if the requested resource is configured to be available on the current partition, if DET error reporting is enabled.
- The Can requires that all variables in NonCacheable MemMap sections be allocated accordingly, to avoid data corruption in multicore context.
- The Can assumes that RTE module implements the EXCLUSIVE AREAS to be core-aware only. The rationale is that the module implementation ensures data integrity by separating the mappable elements for different cores already, thus implementing the EXCLUSIVE AREAS in a blocking manner (ex: spin-lock) on a multicore scope, might affect the performance of the drivers on the two cores, although they might access separate HW elements. For single-core scope, the EXCLUSIVE AREAS keep the same purpose as on previous AUTOSAR implementations. (* - to be updated per Can usecase, to be detailed/removed if some modules require such kind of functionality for critical features which cannot be atomically shared among cores).
- The Can assumes that each interrupt is routed by the system only to the core on which is supposed to be serviced.
- The configuration structure name shall be available in the caller scope of Can_43_FLEXCAN_Init() function by being declared with EXTERN, according to its generated name.

Chapter 6

Main API Requirements

- Main function calls within BSW scheduler
- API Requirements
- Calls to Notification Functions, Callbacks, Callouts

6.1 Main function calls within BSW scheduler

CAN Driver support 4 main functions that can be configured to be scheduled by BSW scheduler: • *void Can_43_FLEXCAN_MainFunction_Write(void)*

- *void Can_43_FLEXCAN_MainFunction_Read(void)*
- *void Can_43_FLEXCAN_MainFunction_BusOff(void)*
- *void Can_43_FLEXCAN_MainFunction_Mode(void)*

These Autosar APIs are scheduled if these 3 events are configured to be in “Polling” mode by the following parameters: • CanTxProcessing

```
#define CAN_43_FLEXCAN_TX_POLLING_SUPPORT (STD_ON)
```

- CanRxProcessing

```
#define CAN_43_FLEXCAN_RX_POLLING_SUPPORT (STD_ON)
```

- CanBusoffProcessing

```
#define CAN_43_FLEXCAN_BUSOFF_POLLING_SUPPORT (STD_ON)
```

The period for polling is configured by the following 4 parameters: • CanMainFunctionWritePeriod

```
#define CAN_43_FLEXCAN_MAINFUNCTION_PERIOD_WRITE (uint32)0.0010U
```

- CanMainFunctionReadPeriod

```
#define CAN_43_FLEXCAN_MAINFUNCTION_PERIOD_READ (uint32)0.0010U
```

- CanMainFunctionBusoffPeriod

```
#define CAN_43_FLEXCAN_MAINFUNCTION_PERIOD_BUSOFF (uint32)0.0010U
```

- CanMainFunctionModePeriod

```
#define CAN_43_FLEXCAN_MAINFUNCTION_MODE_PERIOD (uint32)0.0010U
```

Note

A configuration for an hardware unit can be possible in such a way that one controller will handle events by interrupts and another by polling method.

6.2 API Requirements

SWS_Can_00360,SWS_Can_00361,SWS_Can_00362,SWS_Can_00363,SWS_Can_00228,SWS_Can_00112,SWS_Can_00185,SWS_Can_00235,

6.3 Calls to Notification Functions, Callbacks, Callouts

Call-back Notifications

The CAN stack provides the following call-back notifications:

- *CanIf_TxConfirmation*: This CAN Interface call-back function is called when a CAN message has been transmitted. *void CanIf_TxConfirmation(PduIdType CanTxPduId)*
- *CanIf_RxIndication*: This CAN Interface call-back function is called when valid CAN message is received. *void CanIf_RxIndication(const Can_HwType* Mailbox, const PduInfoType* PduInfoPtr)*
- *CanIf_ControllerBusOff*: This CAN Interface call-back function is called when the CAN controller reached the bus-off state (see CAN specification for further details). *void CanIf_ControllerBusOff(uint8 Controller)*
- *CanIf_ControllerErrorStatePassive*: This CAN Interface call-back function is called when the CAN driver detected the error state passive of CAN controller (see CAN specification for further details). *void CanIf_ControllerErrorStatePassive(uint8 ControllerId, uint16 RxErrorCounter, uint16 TxErrorCounter)*
- *CanIf_ErrorNotification*: This CAN Interface call-back function is called when the CAN driver detected the error state of CAN controller which is predefined in *Can_ErrorType* (see CAN specification for further details). *void CanIf_ErrorNotification(uint8 ControllerId, Can_ErrorType Can_ErrorType)*

User Notification

Chapter 7

Memory allocation

- [Sections to be defined in Can_43_FLEXCAN_MemMap.h](#)
- [Linker command file](#)

7.1 Sections to be defined in Can_43_FLEXCAN_MemMap.h

Section name	Type of section	Description
CAN_43_FLEXCAN_START_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start of Memory Section for Config Data
CAN_43_FLEXCAN_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	End of Memory Section for Config Data
CAN_43_FLEXCAN_START_SEC_CODE	Code	Start of memory Section for Code
CAN_43_FLEXCAN_STOP_SEC_CODE	Code	End of memory Section for Code
CAN_43_FLEXCAN_START_SEC_VARIABLES_CLEARED_UNSPECIFIED	Variables	Used for variables, structures, arrays, when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. These variables are initialized with values after every reset.
CAN_43_FLEXCAN_STOP_SEC_VARIABLES_CLEARED_UNSPECIFIED	Variables	End of above section.
CAN_43_FLEXCAN_START_SEC_VARIABLES_NEVER_CLEARED_UNSPECIFIED	Variables	These variables are never cleared and never initialized by start-up code.
CAN_43_FLEXCAN_STOP_SEC_VARIABLES_NEVER_CLEARED_UNSPECIFIED	Variables	End of above section.
CAN_43_FLEXCAN_START_SEC_CONSTANT_DATA_UNSPECIFIED	Constant Data	Used for constants
CAN_43_FLEXCAN_STOP_SEC_CONSTANT_DATA_UNSPECIFIED	Constant Data	End of above section.
CAN_43_FLEXCAN_START_SEC_VARIABLES_CLEARED_UNSPECIFIED_NO_CACHEABLE	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. Normally, this section is used to store descriptors and data buffers. This section must also be cache inhibited
CAN_43_FLEXCAN_STOP_SEC_VARIABLES_CLEARED_UNSPECIFIED_NO_CACHEABLE	Variables	End of the above section

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>_MemMap.h".



Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section [Files Required for Compilation](#)
2. Allocate the proper memory sections in the driver's memory map header file ("`<Module>_MemMap.h`") and linker command file. For more details refer to section [Sections to be defined in `<Module>\_MemMap.h`](#)
3. Compile & build the module with all the dependent modules. For more details refer to section [Building the Driver](#)

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating the CAN driver into the application.

External Assumption Req ID	External Assumption Text
SWS_Can_00436	Can_GeneralTypes.h shall contain all types and constants that are shared among the AUTOSAR CAN modules Can, CanIf and CanTrcv. Note: Implemented in base
SWS_Can_00415	Name: Can_PduTypeKind: StructureElements: swPduHandleType: PduIdTypeComment: -lengthType: uint8Comment: -idType: Can_IdTypeComment: -sduType: uint8Comment: -Description: This type unites PduId (swPduHandle), SduLength (length), SduData (sdu), and CanId (id) for any CAN L-SDU.Available via: Can_GeneralTypes.h Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00416	Name: Can_IdTypeKind: TypeDerived from: Basetype: Variationuint32: Range: Standard: 0..0x400007FF: 0..0x400007FFExtended: 0..0xDFFF: 0..0xDFFFFFFFDescription: Represents the Identifier of an L-PDU. The two most significant bits specify the frametype: 00 CAN message with Standard CAN ID 01 CAN FD frame with Standard CANID 10 CAN message with Extended CAN ID 11 CAN FD frame with Extended CAN IDAvailable via: Can_GeneralTypes.h Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00429	Name: Can_HwHandleTypeKind: TypeDerived from: Basetype: Variationuint16: -uint8: -Range: Standard: 0..0xFF: 0..0xFFExtended: 0..0xFFFF: 0..0xFFFFDescription: Represents the hardware object handles of a CAN hardware unit. For CAN hardware units with more than 255 HW objects use extended range.Available via: Can_GeneralTypes.h Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00039	Range: CAN_BUSY: 0x02: transmit request could not be processed because no transmit object was availableDescription: Overlayed return value of Std_ReturnType for CAN driver API Can_Write()Available via: Can_GeneralTypes.h Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00024	The valid values that can be configured are hardware dependent. Therefore the rules and constraints can't be given in the standard. The configuration tool is responsible to do a static configuration checking, also regarding dependencies between modules (i.e. Port driver, MCU driver etc.)

External Assumption Req ID	External Assumption Text
SWS_Can_91013	Name: Can_ControllerStateTypeKind: EnumerationRange: CAN_CS← _UNINIT: 0x00: CAN controller state UNINIT.CAN_CS_STARTED← : 0x01: CAN controller state STARTED.CAN_CS_STOPPED: 0x02: CAN controller state STOPPED.CAN_CS_SLEEP: 0x03: CAN controller state SLEEP.Description: States that are used by the several ControllerMode functions.Available via: Can_GeneralTypes.h Note: defined into Can_← GeneralTypes.h included into Base module
SWS_Can_91003	Name: Can_ErrorStateTypeKind: EnumerationRange: CAN_ERR← ORSTATE_ACTIVE: -: The CAN controller takes fully part in communication.CAN_ERRORSTATE_PASSIVE: -: The CAN controller takes part in communication, but does not send active error frames.CAN← _ERRORSTATE_BUSOFF: -: The CAN controller does not take part in communication.Description: Error states of a CAN controller.Available via: Can_GeneralTypes.h Note: defined into Can_GeneralTypes.h included into Base module
SWS_CAN_00496	Name: Can_HwTypeKind: StructureElements: CanIdType: Can_Id← TypeComment: Standard/Extended CAN ID of CAN L-PDUHohType← : Can_HwHandleTypeComment: ID of the corresponding Hardware Object RangeControllerIdType: uint8Comment: ControllerId provided by CanIf clearly identify the corresponding controllerDescription: This type defines a data structure which clearly provides an Hardware Object Handle includ- ing its corresponding CAN Controller and therefore CanDrv as well as the specific CanId.Available via: Can_GeneralTypes.h
SWS_Can_91021	Name: Can_ErrorTypeKind: EnumerationRange: CAN_ERROR_BIT← _MONITORING1: 0x01: A 0 was transmitted and a 1 was read backCAN← _ERROR_BIT_MONITORING0: 0x02: A 1 was transmitted and a 0 was read backCAN_ERROR_BIT: 0x03: The HW reports a CAN bit error but cannot report distinguish between CAN_ERROR_BIT_MONITORING1 and CAN_ERROR_BIT_MONITORING0CAN_ERROR_CHECK_A← CK_FAILED: 0x04: Acknowledgement check failedCAN_ERROR_AC← K_DELIMITER: 0x05: Acknowledgement delimiter check failedCAN_E← RROR_ARBITRATION_LOST: 0x06: The sender lost in arbitration.C← AN_ERROR_OVERLOAD: 0x07: CAN overload detected via an overload frame. Indicates that the receive buffers of a receiver are full.CAN_ERRO← R_CHECK_FORM_FAILED: 0x08: Violations of the fixed frame format← CAN_ERROR_CHECK_STUFFING_FAILED: 0x09: Stuffing bits not as expectedCAN_ERROR_CHECK_CRC_FAILED: 0xA: CRC failed← CAN_ERROR_BUS_LOCK: 0xB: Bus lock (Bus is stuck to dominant level)Description: The enumeration represents a superset of CAN Error Types which typical CAN HW is able to report. That means not all CAN HW will be able to support the complete set.Available via: Can_General← Types.h
EA_RTD_00001	The external application shall call Can_Init function only when the driver state is CAN_UNINIT and the state of all controllers is UNINIT.
EA_RTD_00002	The external application shall call Can_MainFunction_Write only after driver initialization.
EA_RTD_00003	The external application shall call Can_MainFunction_Read only after driver initialization.
EA_RTD_00004	The external application shall call Can_MainFunction_BusOff only after driver initialization.
EA_RTD_00005	The external application shall call Can_MainFunction_Wakeup only after driver initialization.

External Assumption Req ID	External Assumption Text
EA_RTD_00006	The external application shall call Can_SetControllerMode only after driver initialization.
EA_RTD_00007	The external application shall call Can_DisableControllerInterrupts function only after driver initialization.
EA_RTD_00008	The external application shall call Can_EnableControllerInterrupts function only after driver initialization.
EA_RTD_00009	The external application shall call Can_Write function only after driver initialization.
EA_RTD_00010	The external application shall assure that Can_Init does not preempt and is not preempted by any other CAN driver API excepting Can_GetVersionInfo. The external application shall assure that Can_Init does not preempt itself.
EA_RTD_00011	The external application shall assure that Can_MainFunction_Write does not preempt and is not preempted by any other CAN driver API exception Can_GetVersionInfo. The external application shall assure that Can_MainFunction_Write does not preempt itself.
EA_RTD_00012	The external application shall assure that Can_MainFunction_Read does not preempt and is not preempted by any other CAN driver API exception Can_GetVersionInfo. The external application shall assure that Can_MainFunction_Read does not preempt itself.
EA_RTD_00013	The external application shall assure that Can_MainFunction_BusOff does not preempt and is not preempted by any other CAN driver API exception Can_GetVersionInfo. The external application shall assure that Can_MainFunction_BusOff does not preempt itself.
EA_RTD_00014	The external application shall assure that Can_SetControllerMode does not preempt and is not preempted by any other CAN driver API using the same controller parameter. The external application shall assure that Can_SetControllerMode does not preempt itself.
EA_RTD_00015	The external application shall assure that Can_DisableControllerInterrupts does not preempt and is not preempted by any other CAN driver API using the same controller parameter.
EA_RTD_00016	The external application shall assure that Can_EnableControllerInterrupts does not preempt and is not preempted by any other CAN driver API using the same controller parameter.
EA_RTD_00017	The external application shall assure that Can_Write does not preempt and is not preempted by any other CAN driver API using the same controller as the hardware handle parameter. The external application shall assure that Can_Write does not preempt itself for the same hardware handle parameter.
EA_RTD_00019	The external application shall call Can_Init function only when the driver state is CAN_UNINIT and the state of all controllers is UNINIT.
EA_RTD_00020	The external application shall assure that Can_CheckWakeup does not preempt and is not preempted by any other CAN driver API using the same controller parameter. The external application shall assure that Can_CheckWakeup does not preempt itself.
EA_RTD_00021	The external application shall call Can_CheckWakeup function only after driver initialization.
EA_RTD_00022	The external application shall call Can_MainFunction_Mode function only after driver initialization.
EA_RTD_00023	The external application shall call Can_Init function only when the driver state is CAN_UNINIT and the state of all controllers is UNINIT.

External assumptions for driver

External Assumption Req ID	External Assumption Text
EA_RTD_00024	The external application shall call Can_SetControllerMode(CAN_T_STOP) only when the CAN controller is in state STOPPED.
EA_RTD_00025	The external application shall call Can_SetControllerMode(CAN_T_START) only when the CAN controller is in state STARTED or STOPPED.
EA_RTD_00026	The external application shall call Can_SetControllerMode(CAN_T_SLEEP) only when the CAN controller is in state SLEEP or STOPPED.
EA_RTD_00027	The external application shall call Can_SetControllerMode(CAN_T_WAKEUP) only when the CAN controller is in state SLEEP or STOPPED.
EA_RTD_00030	The external application shall assure that Can_CheckBaudrate does not preempt and is not preempted by any other CAN driver API using the same controller parameter.
EA_RTD_00031	The external application shall call Can_CheckBaudrate only after driver initialization.
EA_RTD_00071	If interrupts are locked, a centralized function pair to lock and unlock interrupts shall be used.
EA_RTD_00081	The integrator shall assure that <MSN>_Init() and <MSN>_DeInit() functions do not interrupt each other.
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: Rationale: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00093	The application shall define EcucCoreIDs in a compact/consecutive order, starting from zero.
EA_RTD_00096	The application shall pass the correct initialization pointer, specific to the partition in which the driver is to be used.
EA_RTD_00106	Standalone IP configuration and HL configuration of the same driver shall be done in the same project
EA_RTD_00107	The integrator shall use the IP interface only for hardware resources that were configured for standalone IP usage. Note: The integrator shall not directly use the IP interface for hardware resources that were allocated to be used in HL context.
EA_RTD_00108	The integrator shall use the IP interface to build a CDD, therefore the BSWMD will not contain reference to the IP interface
EA_RTD_00113	When RTD drivers are integrated with AutosarOS and User mode support is enabled, the integrator shall assure that the definition and declaration of all RTD functions needed to be called as trusted functions follow the naming convention Call<Function_Name>TRUSTED(parameter1,parameter2,...) in Integration/User code. They need to be visible in Os.h for the driver to call them. They will call RTD <Function_Name>() as trusted functions in OS specific manner.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.